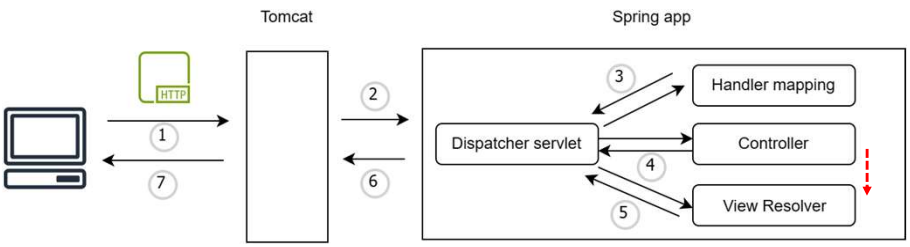
UNIVERSITÀ DI TRENTO

REST services

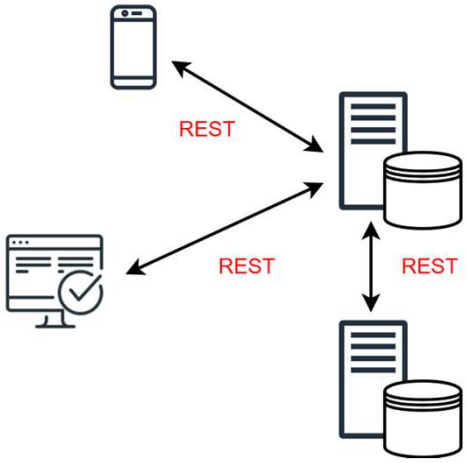
Single process web apps in Spring



- All the functionalities are tightly integrated: e.g. the business logic, the presentation (HTML+ CSS), and DB access are all handled in one place
- That way can make a web app hard to scale and maintain

Implementing REST services

- REST services are one of the most often encountered ways to implement communication between:
  - client-server,
  - two backend components
  - two web apps



3

REST style and RESTful applications

- REST (**RE**presentational **S**tate **T**ransfer) is an architectural style for designing distributed network applications
- REST architectural constraints:
  - Client-Server: Concerns should be separated between clients and servers. Thus, they can evolve independently
  - Stateless: The communication between client and server should be stateless. The server need not remember the state of the client. Instead, clients must include all of the necessary information in the request so that server can understand and process it
  - Layered System: Multiple hierarchical layers such as firewalls, and proxies can exist between client and server. Layers can be added, modified, reordered, or removed transparently to improve scalability

Source: B. Varanasi, S. Belida. Spring REST. Apress, 2015

4

REST style and RESTful applications

- REST architectural constraints:
  - Cache: Responses from the server must be declared as cacheable or noncacheable. This would allow the client or its intermediary components to cache responses and reuse them for later requests. This reduces the load on the server and helps improve the performance
  - Uniform Interface: the interactions between client, server, and intermediary components are based on the uniformity of their interfaces. This simplifies the overall architecture as components can evolve independently as long as they implement the agreed-on contract
  - Code on demand: Clients can extend their functionality by downloading and executing code on demand (e.g., JavaScript scripts). This is an optional constraint

Source: B. Varanasi, S. Belida. Spring REST. Apress, 2015

5

Although, in theory, it is possible to build REST applications using any networking infrastructure or transport protocol, in practice, they leverage features and capabilities of the Web and use HTTP as the transport protocol

A REST application that fully adheres to all the REST architectural constraints is called a RESTful application

6

A REST-like application follows REST conventions but does not strictly satisfy all REST constraints

We will develop REST-like web app

7

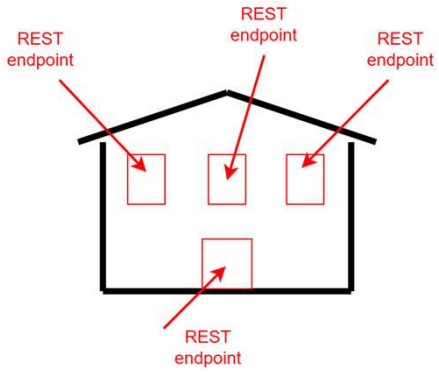
Rest services

- A REST service is a backend module that exposes endpoints for specific resources and follows some REST conventions, such as using standard HTTP methods to operate on these endpoints
- It is not a web app: it provides data and functionality through endpoints, not a user interface.

8

REST endpoints

- A REST endpoint refers to a specific URL exposed by a REST service.
- It is the entry point for client interactions
- Each endpoint corresponds to a unique operation and it is associated with a HTTP method



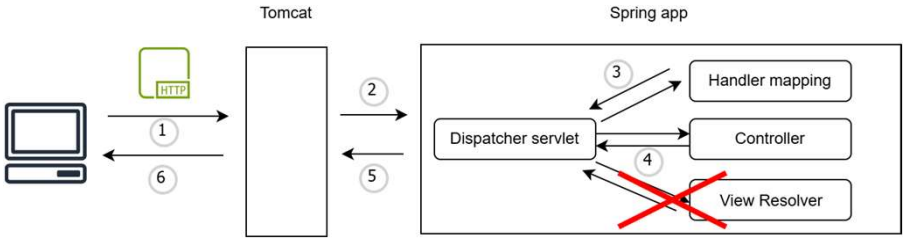
9

REST services in Spring
client-server communication

- In Spring, an application can have both traditional controllers and REST controllers. Traditional controllers have endpoints that return Views (HTML pages), while REST controllers have endpoints that return data in some format
- The response flow changes only for REST controllers: no View Resolver is needed

10

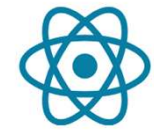
REST services in Spring
client-server communication



- When implementing REST endpoints, the Spring flow changes:
  - A View Resolver is not needed anymore (no page will be built)
  - Controller directly returns data
  - The Dispatcher returns the HTTP response without rendering a view
  - The front-end takes data and render it to the user
- This way, the back-end and the front end are separated
- Much flexibility to work with multiple types of clients (e.g. mobile)

11

Libraries and Framework for UI



React

12

Implementing a REST endpoint

- In Spring, transforming a web controller into a REST controller to implement REST web services is simple!
- Let's see an example:

```
@Controller
public class GreetingsController{

@GetMapping("/ciao")
@ResponseBody
public String ciao(){
return "Ciao!"}

}
```

The @ResponseBody annotation informs the Dispatcher Servlet that this method doesn't return a view name but the data sent directly in the HTTP response

13

Implementing a REST endpoint

- If the controller has other methods (e.g. bonjour) we need to annotate any of them with @ResponseBody

So boring!

- We can do better:

```
@RestController
public class GreetingsController{

@GetMapping("/ciao")
public String ciao(){
return "Ciao!"}

@GetMapping("/bonjour")
public String bonjour(){
return "Bonjour!"}
}
```

To avoid to repeat @RequestBody for each method, we replace @Controller with @RestController

14

Sending data in the response body #1

- What can be returned by a Controller?
- Strings, objects (not only Strings) and a list of objects

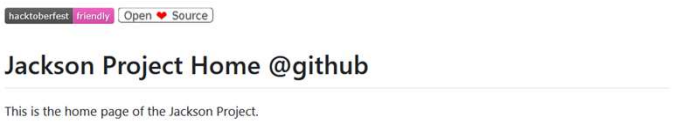
When we use an object to model the data transferred, we name this object a Data Transfer Object (DTO)

- How would the object look in the HTTP response body?

15

Sending data in the response body #2

- We cannot send a raw Java object over the network. We need to convert it in something more appropriate. This is where JSON (XML) comes into play!
- As we studied, Spring Boot automatically sets up all the components needed to handle HTTP requests and responses. Jackson is one of these components
- Jackson is a Java JSON library converting Java objects into JSON and vice versa



esempio21

16

JavaScript Object Notation (JSON)

- *"JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language, Standard ECMA-262 3rd Edition - December 1999. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others. These properties make JSON an ideal data-interchange language."*

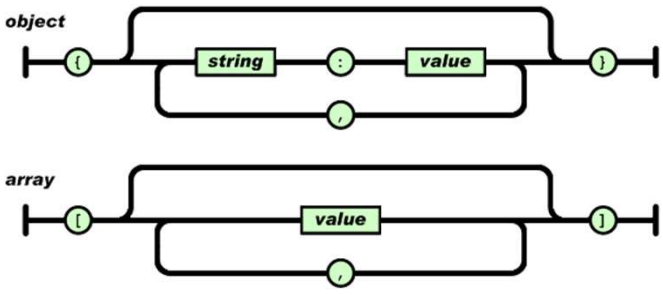
https://www.json.org/json-en.html

- So, JSON in a nutshell:
  - JSON is a lightweight data-interchange format for storing and transport data
  - JSON is a text format
  - JSON is language independent

17

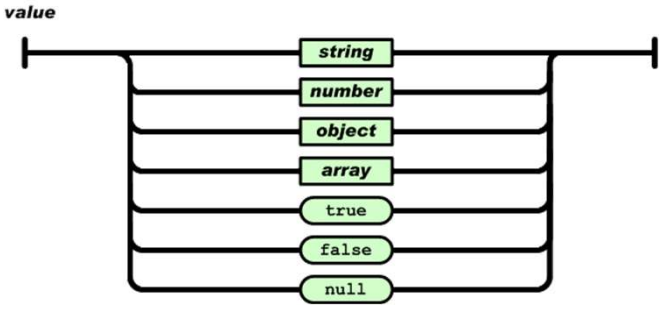
JavaScript Object Notation (JSON)

- JSON is built on two structures:
  - A collection of name-value pairs (object)
  - An ordered list of values (array)



18

JavaScript Object Notation (JSON)



19

JavaScript Object Notation (JSON)

```
{     "name": "Mario",
      "surname": "Rossi",
      "active": true,
      "favoriteNumber": 42,
      "birthday": {
            "day": 1,
            "month": 1,
            "year": 2000
      },
      "languages": [ "it", "en" ]
}
```

20

XML vs. JSON

XML

```
<employees>
  <employee>
    <firstName>John</firstName>
    <lastName>Doe</lastName>
  </employee>
  <employee>
    <firstName>Anna</firstName>
    <lastName>Smith</lastName>
  </employee>
  <employee>
    <firstName>Peter</firstName>
    <lastName>Jones</lastName>
  </employee>
</employees>
```

JSON

```
{"employees":[
  { "firstName":"John",
    "lastName":"Doe" },
  { "firstName":"Anna",
    "lastName":"Smith" },
  { "firstName":"Peter",
    "lastName":"Jones" }
]}
```

21

XML vs JSON

Both JSON and XML:

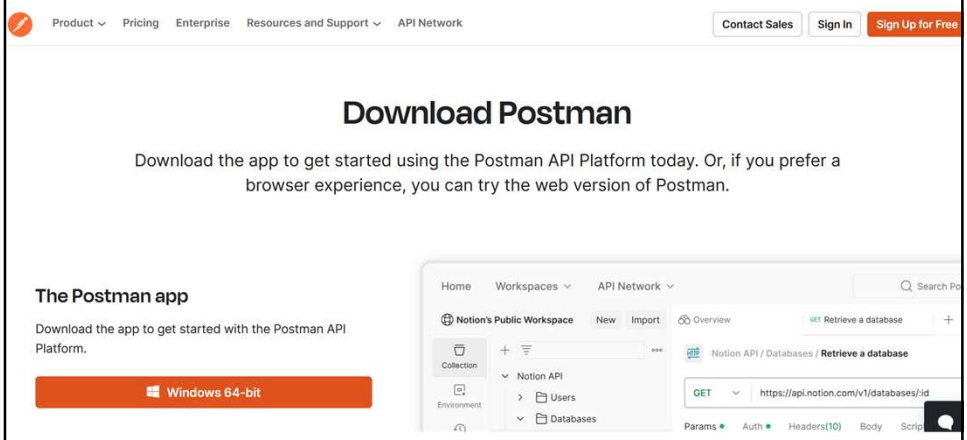
- are "self describing" (human readable)
- are hierarchical (values within values)
- can be parsed and used by lots of programming languages
- can be fetched with an XMLHttpRequest

BUT:

- JSON does not use tags (so it is less verbose than XML)
- JSON can use arrays
- JSON can be parsed by a standard JavaScript function

22

Postman



http://www.postman.com/

23

Great!
But, let's take a step back

24

What happens under the hood

- Spring has the interface HttpMessageConverter that maintains a list of converters that know how to transform Java objects into formats like JSON, XML, and so on
- Spring selects the converter looking at the Accept header in the request. If Accept is missing, Spring tries to determine the format using the Content Negotiation Strategy
- The converter handling JSON, the MappingJackson2HttpMessageConverter, uses the Jackson ObjectMapper that:
  - Inspects the object
  - Reads all its fields and annotations
  - Builds a JSON string from it

25